

Lecture Notes on CS231N

Yiwei Chen



Zhejiang University

Last updated: August 16, 2025

Contents

Lecture 1: History of CV and Introduction to CNNs	3
Lecture 2: Image Classification Pipeline	4
2.1 Goal and Attempts	4
2.2 Classifiers:	4
Lecture 3: Loss Functions and Optimization	7
3.1 Loss Functions	7
3.2 Optimization:	8
3.3 Image features:	10
Lecture 4: Backpropagation and Neural Networks	11
4.1 Backpropagation	11
4.2 Neural Networks:	12
Lecture 5: Convolutional Neural Networks	13
5.1 The Layers of CNNs	13
Lecture 6: Training Convolutional Neural Networks(Part 1)	15
6.1 Activation Functions	15
6.2 Data Preprocessing	18
6.3 Weight Initialization	18
6.4 Batch Normalization	19
6.5 babysitting the learning process	19
6.6 Hyperparameter Optimization	19
Lecture 7: Training Convolutional Neural Networks(Part 2)	21
7.1 Optimization Algorithms	21
7.2 Second-Order Optimization	24
7.3 Model Ensembles	24
7.4 Regularization	24
7.5 Transfer Learning	25
Lecture 8: Deep Learning Frameworks	26
8.1 Deep Learning Framework	26
Lecture 9: CNN Architectures	27
9.1 Case Study: CNN Architectures	27
9.2 Other Architectures	29
Lecture 10: Recurrent Neural Networks	30
10.1 Vanilla RNN	30
10.2 Image Captioning	31
10.3 Long Short-Term Memory (LSTM)	32

Lecture 11: Segmentation, Localization and Detection	34
11.1 Classification and Localization	34
11.2 Object Detection	34
11.3 Instance Segmentation	35
11.4 Semantic Segmentation	36
Lecture 12: Visualizing and Understanding	38
12.1 Some Visualizations	38
12.2 Activating Neurons	38
Lecture 13: Generative Models	41
13.1 Unsupervised Learning	41
13.2 Generative Models	41
Lecture 14: Reinforcement Learning	47
14.1 Markov Decision Process (MDP)	47
14.2 Q-learning	48
14.3 REINFORCE algorithm	48
14.4 Actor-Critic Algorithm	50
14.5 REINFORCE in action	50
Lecture 15: Efficient Inference and Training	51
15.1 Part 1: Algorithms for Efficient Inference	51
15.2 Part 2: Hardware for Efficient Inference	51
15.3 Part 3: Algorithms for Efficient Training	51
15.4 Part 4: Hardware for Efficient Training	51

Lecture 1: History of CV and Introduction to CNNs

- **ImageNet:** Annual competition for image classification, started in 2010.
- **Convolutional Neural Networks (CNNs):** Introduced by Yann LeCun in 1998, CNNs are a type of neural network designed for processing structured grid data, such as images. CNNs show great performance in image classification tasks.

Lecture 2: Image Classification Pipeline

2.1 Goal and Attempts

1. Goal:

The task in Image Classification is to predict a single label (or a distribution over labels as shown here to indicate our confidence) for a given image. Images are 3-dimensional arrays of integers from 0 to 255, of size Width x Height x 3. The 3 represents the three color channels Red, Green, Blue.

2. Attempts:

- Find edges, then corners: does not work well.
- Use large datasets with labels.

2.2 Classifiers:

1. K-Nearest Neighbors (KNN):

- *Description:* When $K = 1$, Find the closest image in the dataset to the input image (Nearest Neighbors (NN)).
- *Distance metric:*
 - (a) L1(Mahanttan) Distance: a squared distance metric.

$$d(x, y) = \sum_i |x_i - y_i| \quad (2.1)$$

- (b) L2(Euclidean) Distance: a squared distance metric.

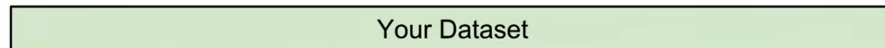
$$d(x, y) = \sqrt{\sum_i (x_i - y_i)^2} \quad (2.2)$$

Rotating the coordinate system changes the L1 distance but not the L2 distance.

- *Performance:*
 - Training time: $O(1)$, as there is nothing to do.
 - Prediction time: $O(N)$, which is inefficient.
- K-Nearest Neighbors (KNN):
 - Description:*
 - When $K = 1$, the classifier is too sensitive to noise.
 - Instead of copying the label of the closest image, take the majority vote of the K closest images.
 - *hyperparameters(超参数):*
 - Choices about the model that are not learned from the data, e.g., K in KNN.
 - To set hyperparameters:*
 - Never use the test set to set hyperparameters.
 - Splitting data into train and test is not enough.
 - **The better idea:** Splitting the training set into training set, validation set, and test set.

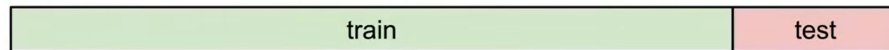
Idea #1: Choose hyperparameters that work best on the data

BAD: $K = 1$ always works perfectly on training data



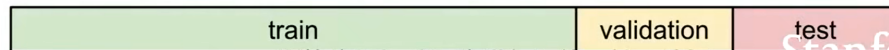
Idea #2: Split data into **train** and **test**, choose hyperparameters that work best on test data

BAD: No idea how algorithm will perform on new data



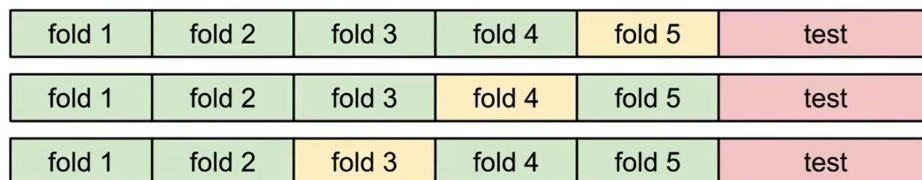
Idea #3: Split data into **train**, **val**, and **test**; choose hyperparameters on val and evaluate on test

Better!



– The common idea: Cross-validation(交叉验证).

Idea #4: Cross-Validation: Split data into **folds**, try each fold as validation and average the results



Useful for small datasets, but not used too frequently in deep learning

- *Pros and Cons:*

Actually, KNN on image is never used:

- Very slow at test time.
- Distance-metrics on pixels are not informative.
- Curse of dimensionality: as the number of dimensions increases, the distance between points becomes less meaningful.

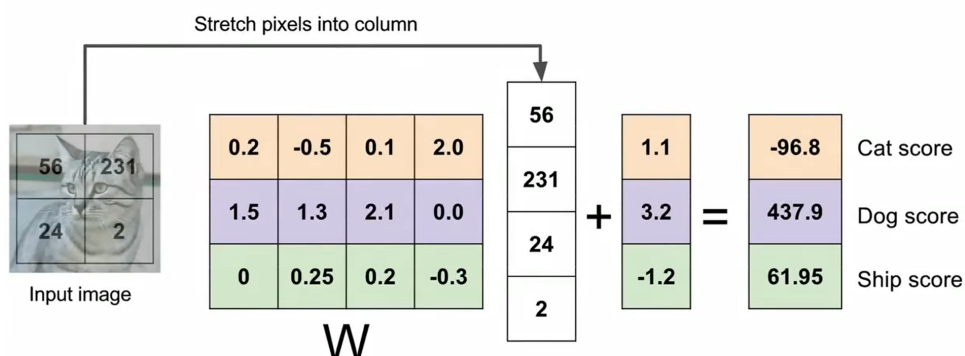
2. Linear Classifier:

- *Description:* A linear classifier makes its predictions based on a linear predictor function combining a set of weights with the feature vector.

$$f(x, W) = Wx + b \quad (2.3)$$

where x is the input image and w is the weight vector, b is the bias term.

And it is important to center the data before applying the linear classifier, namely image data preprocessing.



- *Hard cases:*

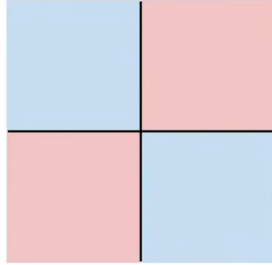
Hard cases for a linear classifier

Class 1:

number of pixels > 0 odd

Class 2:

number of pixels > 0 even

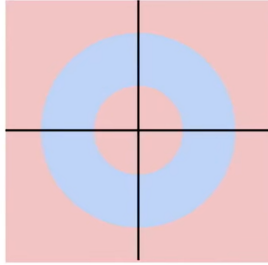


Class 1:

$1 \leq \text{L2 norm} \leq 2$

Class 2:

Everything else

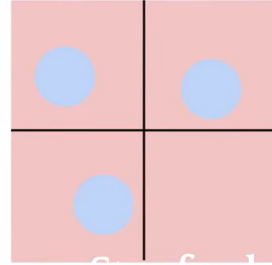


Class 1:

Three modes

Class 2:

Everything else



Lecture 3: Loss Functions and Optimization

3.1 Loss Functions

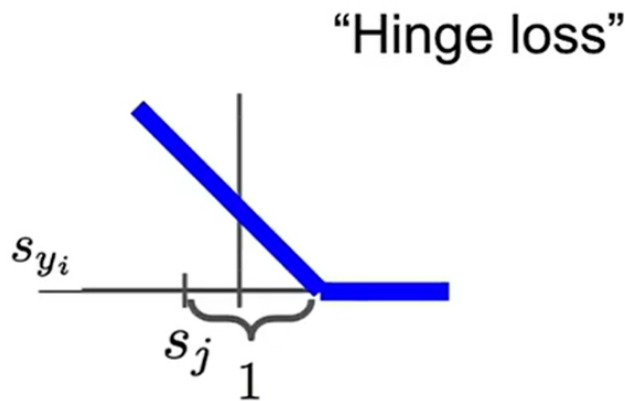
1. Multiple SVM loss:

$$L_i = \sum_{j \neq y_i} \begin{cases} 0 & \text{if } s_{y_i} \geq s_j + 1 \\ s_{y_i} - s_j + 1 & \text{otherwise} \end{cases}$$

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1) = \sum_{j \neq y_i} \max(0, w_j^T x_j - w_{y_i}^T x_{y_i} + 1) \quad (\text{SVM-Loss})$$

where y_i is the label for class i , $s = f(x_i, W)$ is the score for class i , where w_j is the j -th row of W reshaped as a column.

And this kind of loss function is called **Hinge Loss**.



$$L = \frac{1}{N} \sum_{i=1}^N L_i \quad (3.1)$$

For the first training, the W is randomly initialized, so the loss is close to $C - 1$, where C is the number of classes, which can be used to check the correctness of the implementation.

2. regularization:

Furthermore, we can add a regularization term to the loss function to prevent overfitting:

$$L = \frac{1}{N} \sum_{i=1}^N L_i + \lambda R(W) = \frac{1}{N} \sum_{i=1}^N \sum_{j \neq y_i} \max(0, f(x_i, W)_j - f(x_i, W)_{y_i} + \Delta) + \lambda R(W) \quad (3.2)$$

Here Δ actually does not matter. It turns out that this hyperparameter can safely be set to $\Delta = 1.0$ in all cases.

The hyperparameters Δ and λ seem like two different hyperparameters, but in fact they both control the same tradeoff: The tradeoff between the data loss and the regularization loss in the objective.

Therefore, the exact value of the margin between the scores is in some sense meaningless because the weights can shrink or stretch the differences arbitrarily. Hence, the only real tradeoff is how large we allow the weights to grow (through the regularization strength λ).

In common use:

- **L2 regularization:** $R(W) = \|W\|^2$
- L1 regularization: $R(W) = \|W\|_1$
- Elastic net: $R(W) = \beta\|W\|^2 + \|W\|_1$
- Max norm regularization: might see later
- Dropout: might see later
- Fancier: Batch normalization, stochastic depth, etc.

3. Softmax Classifier(Multinomial Logistic Regression):

scores = unnormalized log probabilities of the classes.

$$P(y_i|x_i) = \frac{e^{s_{y_i}}}{\sum_{j=1}^C e^{s_j}} \quad \text{where } C \text{ is the number of classes} \quad (3.3)$$

$$L_i = -\log P(y_i|x_i) = -\log \frac{e^{s_{y_i}}}{\sum_{j=1}^C e^{s_j}} = -s_{y_i} + \log \sum_{j=1}^C e^{s_j} \quad (3.4)$$

For Softmax, even though the right score is much higher than all the other scores, the loss is still not zero, which is different from SVM.

Here, it's actually a cross-entropy loss function, which is defined as follows:

$$H(p, q) = - \sum_x p(x) \log q(x) \quad (\text{Cross-entropy})$$

where p is the "true" distribution and q is the predicted distribution.

For Softmax, $q = e^{s_j} / \sum_{k=1}^C e^{s_k}$, and the "true" distribution is a one-hot vector($p = [0, \dots, 1, \dots, 0]$ contains a single 1 at the y_i -th position).

When the score is big, overflow may happen, so we need to subtract the maximum score from all scores before exponentiating:

```
shifted_logits = x - np.max(x, axis=1, keepdims=True)
```

3.2 Optimization:

Strategy: follow the slope

So first we need to compute the gradient of the loss function with respect to the weights W .

The method of finite differences($(f(W+h) - f(W))/h$) is bad because of large amount of computation, don't use it to train but to **check the correctness** of the implementation.

step size/learning rate: a hyperparameter that controls how much to change the model in response to the estimated gradient.

1. Derivative of the loss function:

$$\nabla_W L(W) = \frac{1}{N} \sum_{i=1}^N \nabla_W L_i(x_i, y_i, W) + \lambda \nabla_W R(W) \quad (3.5)$$

The scores of each class are influenced by the rows of W corresponding to the classes.

Hence, the gradient of the loss function with respect to the weights W should focus on the rows. (assuming W is $(C * D)$)

- **For Hinge Loss:**

$$L_i = \sum_{j \neq y_j} [\max(0, w_j^T x_i - w_{y_i}^T x_i + \Delta)]$$

where w_j is the j -th row of W reshaped as a column.

Taking the gradient with respect w_{y_i}

$$\nabla_{w_{y_i}} L_i = -(\sum_{j \neq y_j} 1(w_j^T x_i - w_{y_i}^T x_i + \Delta > 0)) x_i \quad (3.6)$$

where 1 is the indicator function that is one if the condition inside is true or zero otherwise.

For the other rows where $j \neq y_i$ (for every j) the gradient is:

$$\nabla_{w_j} L_i = 1(w_j^T x_i - w_{y_i}^T x_i + \Delta > 0) x_i \quad (3.7)$$

- **For Softmax Loss:**

$$L_i = -\log P(y_i | x_i) = -\log \frac{e^{s_{y_i}}}{\sum_{j=1}^C e^{s_j}} = -s_{y_i} + \log \sum_{j=1}^C e^{s_j}$$

Taking the gradient with respect w_{y_i}

$$\nabla_{w_{y_i}} L_i = -x_i + \frac{e^{s_{y_i}}}{\sum_{j=1}^C e^{s_j}} x_i = -x_i + P(y_i | x_i) x_i \quad (3.8)$$

For the other rows where $j \neq y_i$ (for every j) the gradient is:

$$\nabla_{w_j} L_i = \frac{e^{s_j}}{\sum_{k=1}^C e^{s_k}} x_i = P(j | x_i) x_i \quad (3.9)$$

2. Stochastic Gradient Descent(on-line gradient descent):

Full sum computation is too expensive, so we can use mini-batch(32/64/128) gradient descent.

Every iteration, we randomly sample a mini-batch of N training examples from the training set, and compute the gradient of the loss function with respect to the weights W using only this mini-batch.

SGD technically refers to using a single example at a time to evaluate the gradient.

However, you will hear people use the term SGD even when referring to mini-batch gradient descent (i.e. mentions of MGD for “Minibatch Gradient Descent”, or BGD for “Batch gradient descent” are rare to see), where it is usually assumed that mini-batches are used.

MGD: using a small batch of examples to evaluate the gradient.

BGD: using the entire training set to evaluate the gradient.

3.3 Image features:

Some times, only using the raw pixel values is not enough, we need to extract some features from the image.

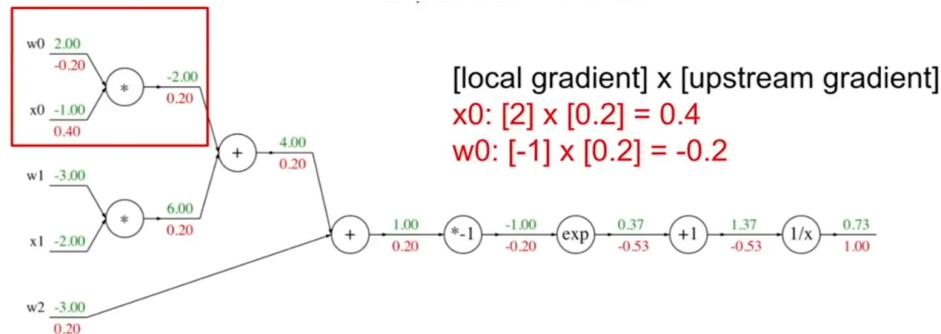
For example:

- color histogram(直方图): a representation of the distribution of colors in an image.
- HOG(Histogram of Oriented Gradients): a feature descriptor focusing on the edges and contours of objects in an image.
- Bag of Words: a representation of an image as a collection of local features, where each feature is represented by a visual word.

Lecture 4: Backpropagation and Neural Networks

4.1 Backpropagation

1. Computation Graph:



Computation Graph Example: $f(x) = \frac{1}{1+e^{-(\omega_0 x_0 + \omega_1 x_1 + \omega_2 x_2)}}$

Along the edges of the graph, we do **forward** computations to get the scores of the classes, and get the output label.

And **backward** computations to get the gradients of the loss function with respect to the weights W and the input x .

2. Backpropagation

Sigmoid function:

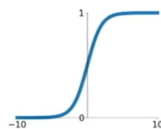
$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (4.1)$$

It is a common activation function used in neural networks, especially in the output layer for binary classification tasks.

Activation functions

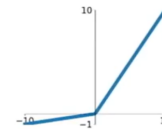
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



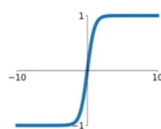
Leaky ReLU

$$\max(0.1x, x)$$



tanh

$$\tanh(x)$$

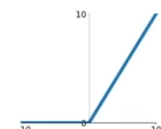


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

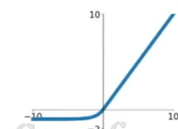
ReLU

$$\max(0, x)$$



ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Activation Functions

- **Add gate:** gradient distributor
- **Max gate:** gradient router
only the maximum value will pass through the gate, and the others will be zero.
- **Mul gate:** gradient switcher

Additionally, if the element is a vector, the gradient will be Jacobian matrix with the same shape as the input.

For $f(x, W) = Wx + b$ (x is a batch), the gradient is:

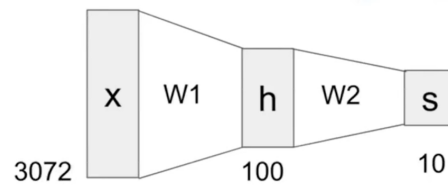
$$\nabla_x = dout * W^T, \quad \nabla_W = x^T * dout, \quad \nabla_b = \sum dout * 1 \quad (4.2)$$

4.2 Neural Networks:

Neural networks: without the brain stuff

(Before) Linear score function: $f = Wx$

(Now) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$

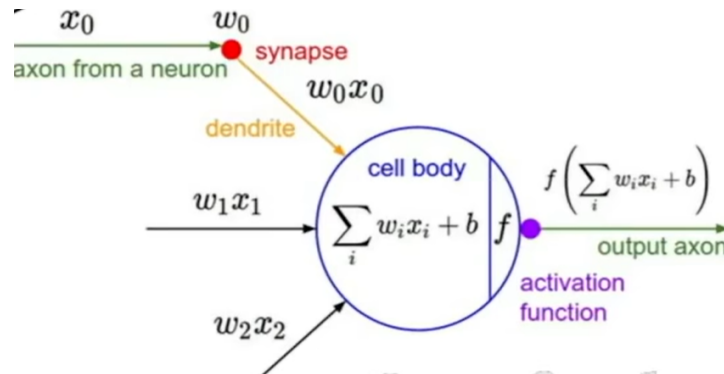


Neural Network Example: $f(x) = W_2 \max(0, W_1 x)$

Here, W_1 contains the weights for the first layer, which can be thought of as computing the scores of the templates.

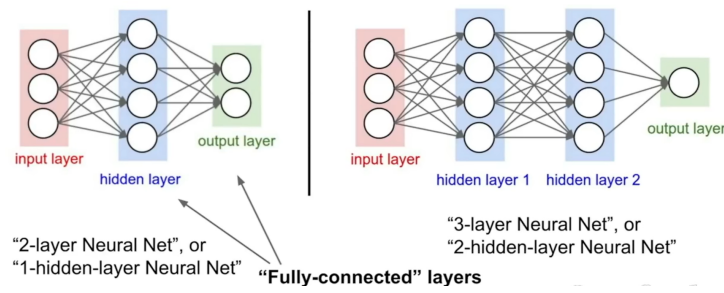
And W_2 contains the weights for the second layer, which can be thought of as computing the final scores for the classes by taking a weighted sum of the scores from the first layer.

Below are figures showing the structure of a neural network.



Neuron-like structure

Neural networks: Architectures



Neural Network Architecture

Lecture 5: Convolutional Neural Networks

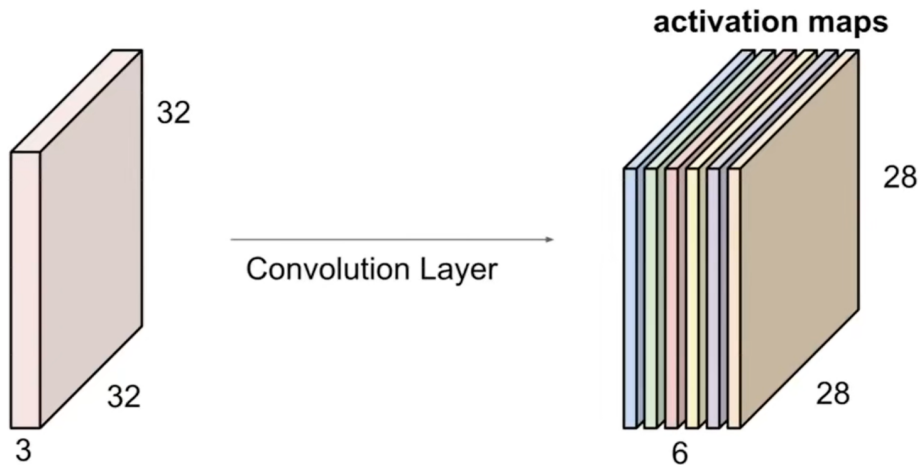
5.1 The Layers of CNNs

1. Convolution Layer

Instead of stretching the input image into a long vector, now preserve the spatial structure.

- **Filter:** Filters always extend the full **depth** of the input volume.

Convolve(slide) over all spatial locations. Each filter is kind of looking for a specific feature in the image, so usually we have many filters in a convolutional layer.



Convolution Operation

- **Convolution:**

In mathematical terms, the convolution operation is defined as:

$$f[x, y] * g[x, y] = \sum_{n_1=-\infty}^{\infty} \sum_{n_2=-\infty}^{\infty} f[n_1, n_2] * g[x - n_1, y - n_2] \quad (5.1)$$

which means that the filter is flipped both horizontally and vertically before being applied to the image.

What we commonly do is a cross-correlation(互相关) operation, without flipping the filter, but we call it convolution.

Output size = $(W - F + 2P)/S + 1$, where W is the input size, F is the filter size, P is the padding size, and S is the stride size.

Dilated convolutions: a convolution operation where the filter is applied to the input with gaps between the filter elements, allowing for a larger receptive field without increasing the number of parameters.

2. Pooling Layer

makes the representation smaller and more manageable.

Similar to convolution layer, strides are also used in pooling layers, but padding is not commonly used.

- **Max Pooling**
- **General Pooling:** average pooling or even L2-norm pooling are also used.

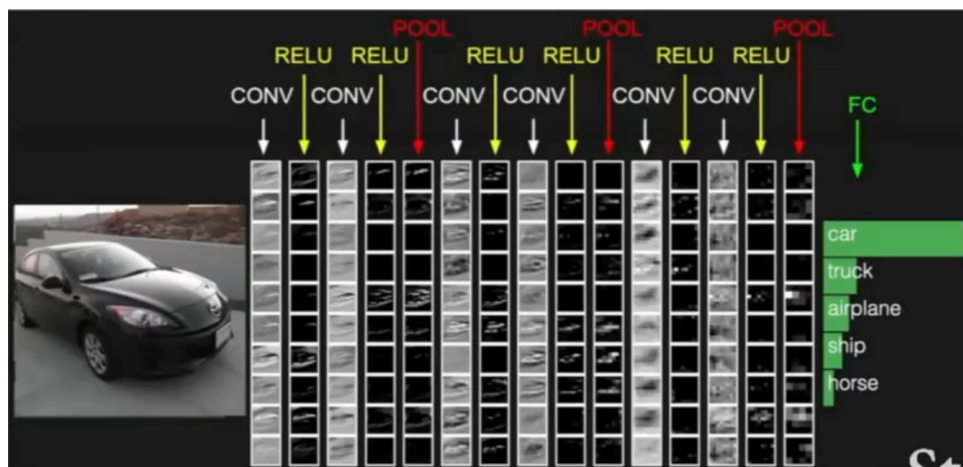
3. Fully Connected Layer

At the end of the network, we have a fully connected layer, which is just a normal neural network layer.

4. Normalization Layer

Many types of normalization layers have been proposed for use in ConvNet architectures, sometimes with the intentions of implementing inhibition schemes observed in the biological brain. However, these layers have since fallen out of favor because in practice their contribution has been shown to be minimal, if any.

In summary, the steps are like this:



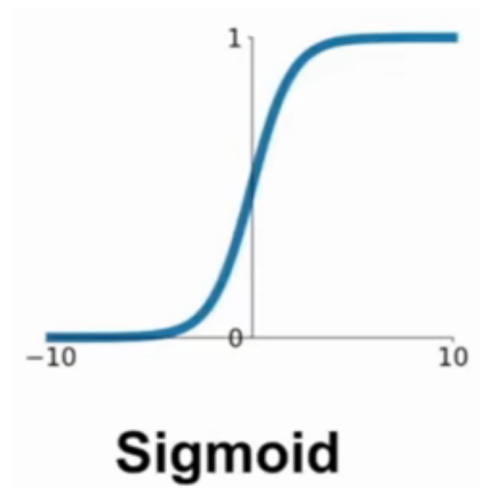
Steps of Convolutional Neural Networks

Lecture 6: Training Convolutional Neural Networks(Part 1)

6.1 Activation Functions

1. Sigmoid:

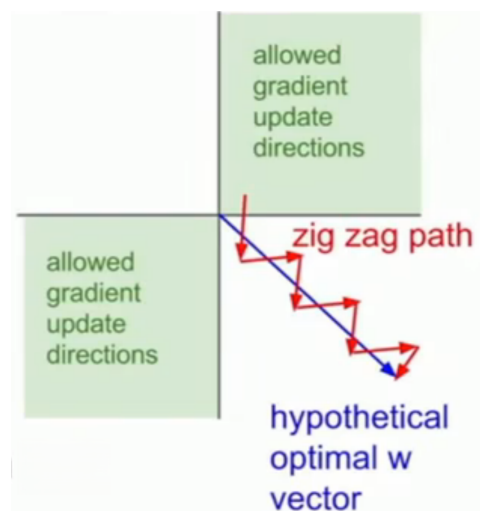
$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (6.1)$$



Sigmoid Activation Function

Three problems with sigmoid:

- (a) Vanishing gradient problem: when the input is too large or too small, the gradient becomes very small, which slows down the training process.
- (b) Output is not zero-centered: the output of the activation function is always positive, which means the gradients ($\partial L / \partial \omega = \partial L / \partial f \cdot \partial f / \partial \omega = \delta \cdot x$, in which f is the local gradient including the activation function) are always positive or negative according to $\partial L / \partial \omega$, which can lead to inefficient updates:



Sigmoid Output is not Zero-Centered

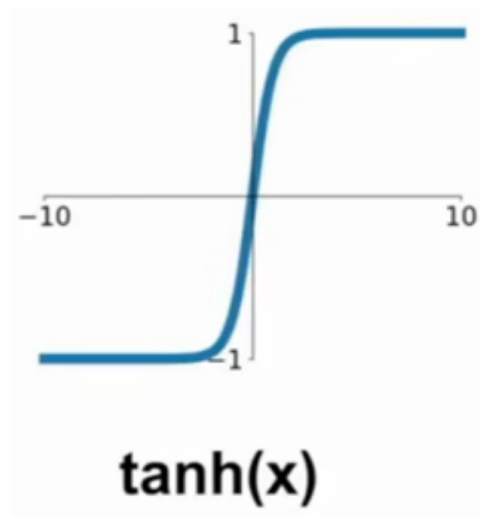
For example, if the optimal direction is the blue arrow, since positive or negative gradients only, the zig zag path is taken, which is inefficient.

(c) $\exp()$ is computationally expensive.

2. **tanh:**

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{2}{1 + e^{-2x}} - 1 \quad (6.2)$$

It gets rid of the not zero-centered problem compared to sigmoid, but still has the vanishing

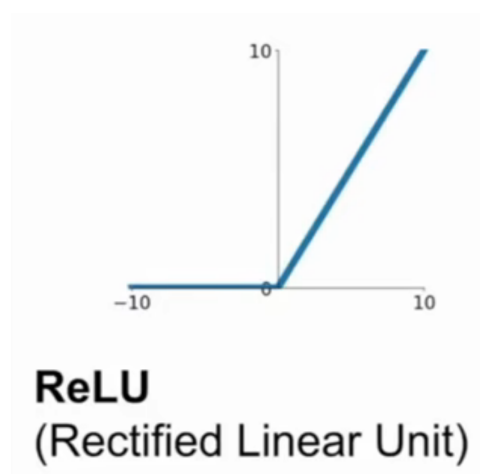


Tanh Activation Function

gradient problem.

3. **ReLU(Rectified Linear Unit):**

$$\text{ReLU}(x) = \max(0, x) \quad (6.3)$$



ReLU Activation Function

- Does not saturate (gradient equals 0) in the positive domain, so it does not suffer from the vanishing gradient problem.
- Computationally efficient: only requires a simple thresholding at zero.
- Converges much faster than sigmoid or tanh.
- More biologically plausible: it is similar to the firing rate of a neuron, which is zero when the input is negative and increases linearly with the input when it is positive.

However, it is still not zero-centered.

Dead ReLU: when the input is negative, the output is always zero, which means the gradient is always zero, and the neuron is "dead": never update.

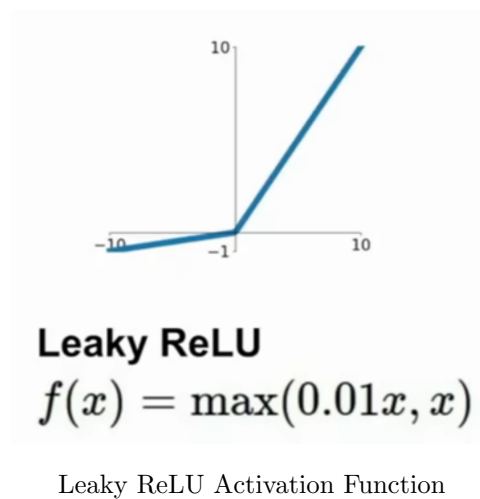
People like to initialize ReLU with a small positive bias(0), so that the neuron is not dead at the beginning.

4. Leaky ReLU:

Parametric ReLU:

$$\text{Parametric ReLU}(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{if } x \leq 0 \end{cases} \quad (6.4)$$

where α is a small positive constant, usually set to 0.01.



On the base of ReLU:

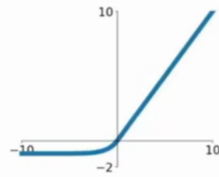
- it does not saturate.
- it will not die.

5. Exponential Linear Unit (ELU):

$$\text{ELU}(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha(e^x - 1) & \text{if } x \leq 0 \end{cases} \quad (6.5)$$

- All benefits of ReLU.
- Closer to zero mean output.
- Negative saturation region: add some robustness to noise.

Exponential Linear Units (ELU)



ELU Activation Function

- Use **ReLU**. Be careful with your learning rates
- Try out **Leaky ReLU / Maxout / ELU**
- Try out **tanh** but don't expect much
- **Don't use sigmoid**

Suggestions for Choosing Activation Functions

6. Maxout neuron:

$$\text{Maxout}(x) = \max(\omega_1^T x + b_1, \omega_2^T x + b_2) \quad (6.6)$$

Linear Regime. Does not saturate. Does not die.

However, double the number of parameters compared to ReLU.

Some suggestions for choosing activation functions:

6.2 Data Preprocessing

1. Zero centering: For [32, 32, 3] images:

- Subtract the mean image(AlexNet): [32, 32, 3].
- Subtract the mean of each channel(VGGNet): mean of each 3 channels.

2. Normalization: For image data, we actually only do zero centering, since the pixel values are already in the range of 0 to 255, which is not too large.

6.3 Weight Initialization

1. **Zero initialization:** all neurons will be the same, and the gradients will be the same, so all neurons will learn the same features, which is not good.
2. **Small random initialization:** usually use a Gaussian distribution with mean 0 and small standard deviation, e.g., 0.01.

The problem is in deep networks, Wx becomes smaller and smaller, all the activation values will shrink to zero, then the gradients will also shrink to zero.

3. **Normal Gaussian Initialization:** problem: almost all the neurons completely saturate, so the gradients are almost zero.

4. **Xavier Initialization:** for tanh and sigmoid, we want the variance of the output to be the same as the variance of the input, so we set the variance of the weights to be $1/n$, where n is the number of input neurons.

6.4 Batch Normalization

1. compute the **empirical** mean and variance independently for each dimension. not computed based on each batch.
2. normalize:

$$\hat{x}^{(k)} = \frac{x^{(k)} - E[x^{(k)}]}{\sqrt{Var[x^{(k)}]}} \quad (6.7)$$

Usually inserted after Fully Connected or Convolutional layers, before the activation function.

3. scale and shift:

$$y^{(k)} = \gamma \hat{x}^{(k)} + \beta \quad (6.8)$$

where γ and β are learnable parameters, which

- improves gradient flow through the network.
- allows higher learning rates.
- reduces the strong dependence on initialization.
- acts as a regularizer, reducing the need for Dropout.

6.5 babysitting the learning process

1. Preprocess the data
2. Choose the architecture

6.6 Hyperparameter Optimization

learning rate: $1e-5$ $1e-3$

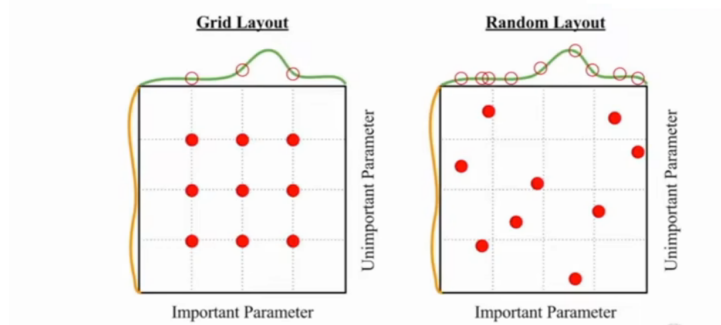
consider using log space, but uniform space.

1. first stage: only a few epochs to get rough idea of what params work.
2. second stage: longer running time, finer search.

when the cost get 3x larger than the original cost, break out.

Consider random search instead of grid search, since it is more efficient and can cover a larger space.

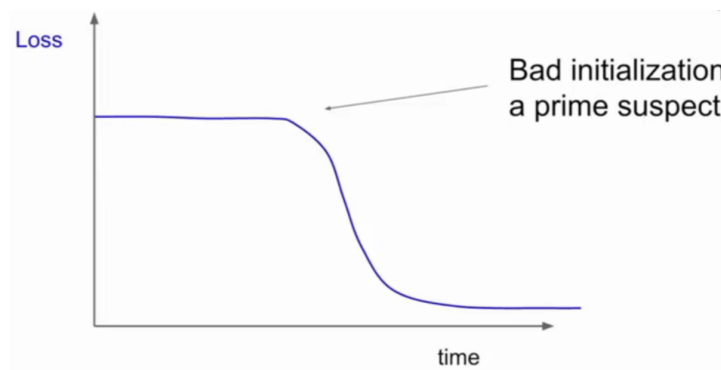
Random Search vs. Grid Search



Hyperparameter Search

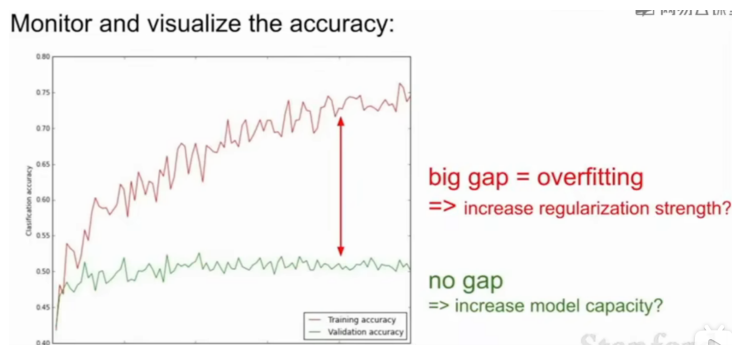
Some experience:

- bad initialization.



loss curve is flat then suddenly decreasing

- overfit or underfit.



Overfitting and Underfitting

- ratio between the updates and values: around 0.001.

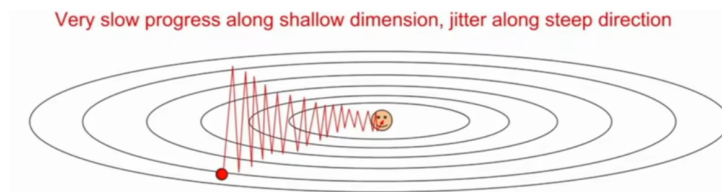
Lecture 7: Training Convolutional Neural Networks(Part 2)

7.1 Optimization Algorithms

1. Problems with SGD

- (a) Poor conditioning: Follow a zig-zag path to convergence, reducing efficiency.

Poor conditioning essentially happens when the loss function has very different curvature in different directions (the Hessian's largest and smallest eigenvalues differ greatly, i.e., high condition number)



p1

- (b) saddle point/local minima



Saddle Point

- (c) noisy gradients

every time only a small batch is used to compute the gradient other than the full training dataset, leading to high variance in the updates.

2. SGD + Momentum

- SGD + Momentum

build up "velocity" as mean of gradients with weight decay:

SGD	SGD+Momentum
$x_{t+1} = x_t - \alpha \nabla f(x_t)$	$v_{t+1} = \rho v_t + \nabla f(x_t)$ $x_{t+1} = x_t - \alpha v_{t+1}$
<pre>while True: dx = compute_gradient(x) x += learning_rate * dx</pre>	<pre>vx = 0 while True: dx = compute_gradient(x) vx = rho * vx + dx x += learning_rate * vx</pre>

SGD with Momentum

- Nesterov Momentum

Nesterov momentum is a variant of momentum that incorporates the gradient of the loss function at the "lookahead" position, which can lead to better convergence properties.

Nesterov Momentum

$$v_{t+1} = \rho v_t - \alpha \nabla f(x_t + \rho v_t)$$

$$x_{t+1} = x_t + v_{t+1}$$

Annoying, usually we want update in terms of $x_t, \nabla f(x_t)$

Change of variables $\tilde{x}_t = x_t + \rho v_t$ and rearrange:

$$v_{t+1} = \rho v_t - \alpha \nabla f(\tilde{x}_t)$$

$$\tilde{x}_{t+1} = \tilde{x}_t - \rho v_t + (1 + \rho)v_{t+1}$$

$$= \tilde{x}_t + v_{t+1} + \rho(v_{t+1} - v_t)$$

```
dx = compute_gradient(x)
old_v = v
v = rho * v - learning_rate * dx
x += -rho * old_v + (1 + rho) * v
```

Nesterov Momentum

3. Adagrad and RMSprop

- Adagrad

Adagrad is an adaptive learning rate optimization algorithm that adjusts the learning rate for each parameter based on the historical gradients. It gives larger updates for infrequent parameters and smaller updates for frequent parameters.

However, the learning rate shrinks continuously, which can lead to premature convergence. Therefore, it is often not used in practice.

AdaGrad

```
grad_squared = 0
while True:
    dx = compute_gradient(x)
    grad_squared += dx * dx
    x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```

Adagrad

- RMSprop

Based on Adagrad, RMSprop maintains a moving average of the squared gradients to normalize the gradients. This helps to mitigate the problem of vanishing or exploding gradients.

RMSProp

```
grad_squared = 0
while True:
    dx = compute_gradient(x)
    grad_squared = decay_rate * grad_squared + (1 - decay_rate) * dx * dx
    x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```

RMSprop

4. Adam

Adam (Adaptive Moment Estimation) is an optimization algorithm that combines the benefits of both Adagrad and RMSprop. It maintains a moving average of both the gradients and the squared gradients, which helps to adaptively adjust the learning rates for each parameter.

Adam (almost)

网易云课堂

```
first_moment = 0
second_moment = 0
while True:
    dx = compute_gradient(x)
    first_moment = beta1 * first_moment + (1 - beta1) * dx
    second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
    x -= learning_rate * first_moment / (np.sqrt(second_moment) + 1e-7)
```

Momentum

AdaGrad / RMSProp

Sort of like RMSProp with momentum

Adam

Since initializing the second moment to 0, the first step can be very large, which can lead to instability in the optimization process. To counteract this, Adam includes a bias correction term that adjusts the learning rates based on the moving averages of the gradients and squared gradients.

Hence bias is used:

Adam (full form)

```

first_moment = 0
second_moment = 0
for t in range(num_iterations):
    dx = compute_gradient(x)
    first_moment = beta1 * first_moment + (1 - beta1) * dx
    second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
    first_unbias = first_moment / (1 - beta1 ** t)
    second_unbias = second_moment / (1 - beta2 ** t)
    x -= learning_rate * first_unbias / (np.sqrt(second_unbias) + 1e-7)

```

Momentum

Bias correction

AdaGrad / RMSProp

Bias correction for the fact that first and second moment estimates start at zero

Adam with beta1 = 0.9, beta2 = 0.999, and learning_rate = 1e-3 or 5e-4 is a great starting point for many models!

Adam with Bias Correction

7.2 Second-Order Optimization

Second-order optimization methods use information about the curvature of the loss function to make more informed updates to the model parameters.

L-BFGS: need to compute the Hessian matrix, which can be expensive for large models.

7.3 Model Ensembles

To reduce training error, combines several models to improve overall performance.

7.4 Regularization

1. Dropout:

In each forward pass, randomly set some neurons (activation) to zero with a probability p , which is a hyperparameter.

- Add randomnessForce the model to learn more robust features that are useful in conjunction with other neurons, rather than relying on specific ones.
- average out the randomness at test-time at test time

Dropout mask: A 0/1 matrix to decide whether to drop a neuron or not.

$$\text{mask} = (\text{np.random.rand}(*\text{x.shape}) \geq p) / (1 - p)$$

Here, divide by $1 - p$ to maintain the expectation of the output between training and testing phases.

2. Batch Normalization

3. Data Augmentation:

Random mix of translation/rotation/stretching/shearing/lens distortions...

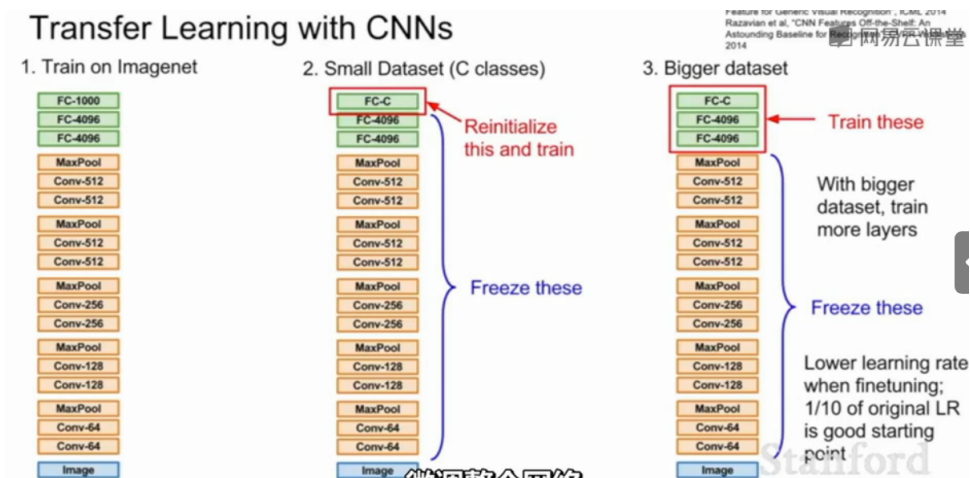
4. Fractional Max Pooling

5. Stochastic Depth:

Randomly drop entire layers during training, which helps to prevent overfitting and encourages the model to learn more robust features.

7.5 Transfer Learning

Transfer learning is a technique where a model trained on one task is fine-tuned on a different but related task.



Transfer Learning

Lecture 8: Deep Learning Frameworks

8.1 Deep Learning Framework

1. Pytorch

Every time build a computational graph, and then compute the gradients using backpropagation(Dynamic Computational Graph).

Use a different data type(e.g. `torch.cuda.FloatTensor`) to adapt to GPU.

- Variable is a node in a computational graph. `requires_grad=True` is used to track gradients.
- `x.data` is a Tensor.
- `x.grad` is a Variable of gradients.
- `x.grad.data` is a Tensor of gradients.

Use `nn` to encapsulate layers and models.

Visdom: something like TensorBoard.

2. TensorFlow

- Build computational graphs first(Static Computational Graph). TensorFlow Fold allows for dynamic computation graphs.
- Automatic differentiation.

Keras: High-Level Wrapper

TensorBoard: Visualization tool for TensorFlow.

Lecture 9: CNN Architectures

9.1 Case Study: CNN Architectures

1. LeNet

One of the earliest CNN architectures, just the simple ones above.

2. AlexNet

A deeper CNN architecture with more Layers and filters.

3. VGGNet

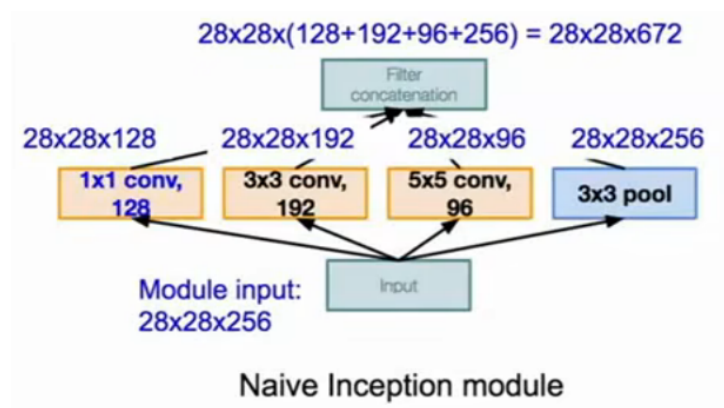
A CNN architecture that emphasizes depth, using very small (3x3) convolution filters.

Stack of three 3x3 convolutional layers has same receptive field as a single 7x7 convolutional layer, but with fewer parameters.

4. GoogleNet

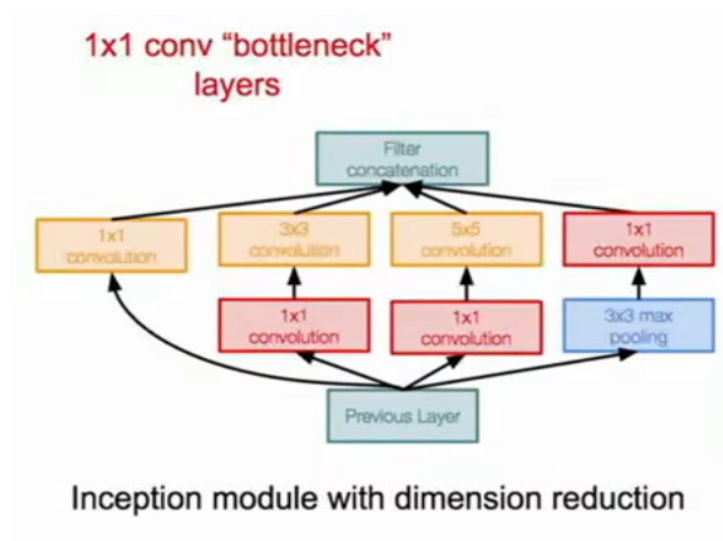
A CNN architecture that introduces the Inception module, which allows for multi-scale feature extraction by concatenating filters of different sizes.

No full connection layers.



Naive Inception Module

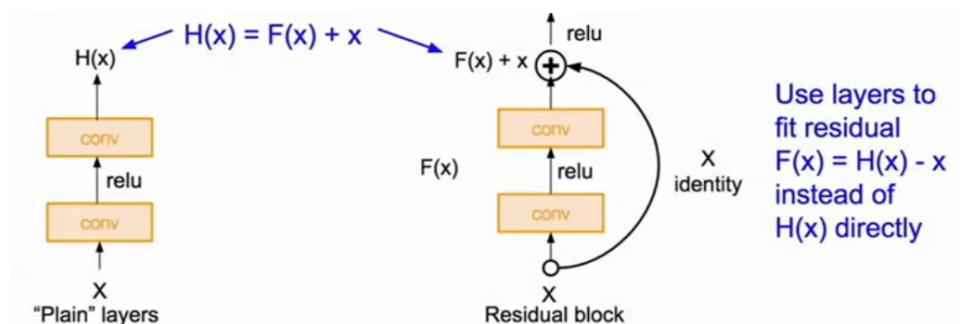
Since the depth is increased, the dimension reduction is needed using 1x1 "bottleneck" layers:



Inception Module

5. ResNet

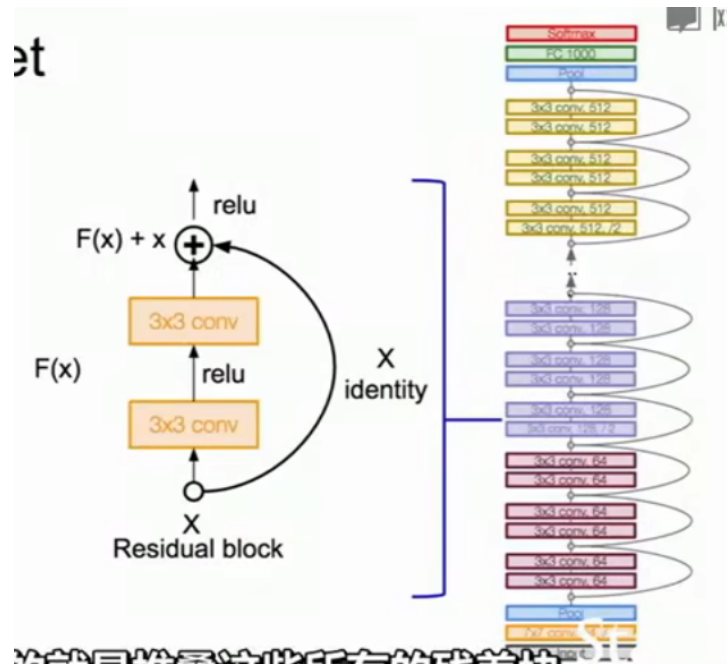
Too deeper CNN architecture seems not to learn well, hence, the input itself is also transformed through a series of convolutions and identity mappings, allowing for better gradient flow and feature reuse.



Residual Block

where $F(x)$ is the residual function and x is the input.

"Bottleneck" layers are also considered.



ResNet Architecture

9.2 Other Architectures

1. Network in Network (NiN)

Using multilayer perceptron (MLP) layers (FC, i.e. 1x1 convolution)

2. Improving ResNets:

- Identity Mappings in Deep Residual Networks
- Wide Residual Networks($F \times k$ filters)
- Aggregated Residual Transformations
- Deep Networks with Stochastic Depth(drop out)

3. Beyond ResNets:

- FractalNet

A CNN architecture that uses a fractal-like pattern to increase depth and improve feature extraction.

- Densely Connected Convolutional Networks (DenseNet)

A CNN architecture that connects each layer to every other layer in a feed-forward fashion, encouraging feature reuse and improving gradient flow.

4. SqueezeNet

A CNN architecture that aims to achieve a smaller model size while maintaining accuracy, using techniques like fire modules and squeeze layers.

Lecture 10: Recurrent Neural Networks

From one-to-one (Vanilla Neural Network) to one-to-many, many-to-one, many-to-many.

10.1 Vanilla RNN

$$h_t = f_W(h_{t-1}, x_t)$$

↓

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$
$$y_t = W_{hy}h_t$$

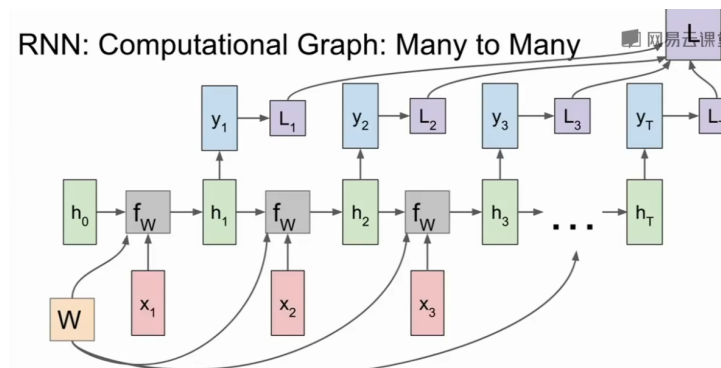
Vanilla RNN

The weight W is reused, hence in backpropagation, the sum of the gradients is computed over all time steps.

x: input layer; h: hidden layer; y: output layer.

The computation graph is as follows:

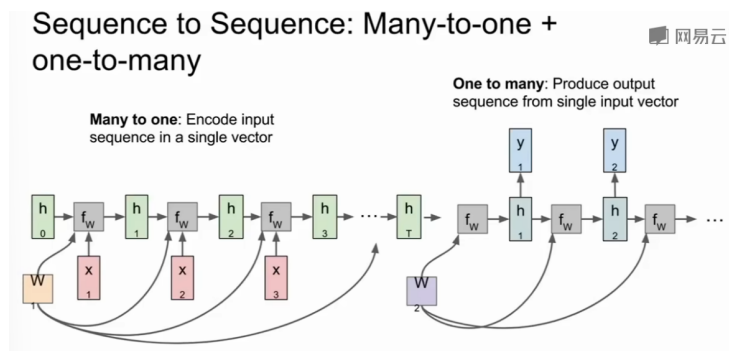
- many-to-many:



Many-to-Many RNN

- sequence to sequence: many-to-one + one-to-many:

Encoder + Decoder:



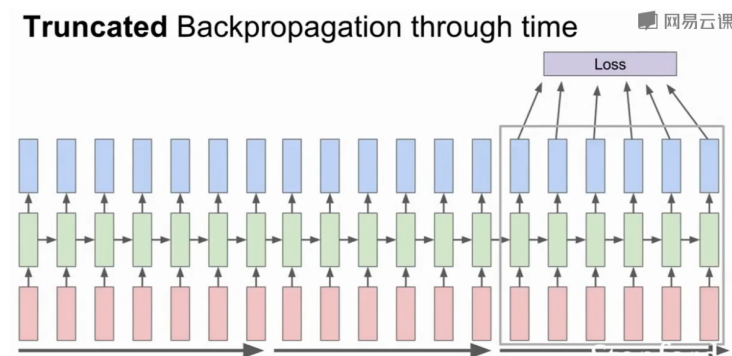
Sequence to Sequence

Backpropagation through time (BPTT):

When the sequence is long, the gradients can vanish or explode, which makes it hard to train the model.

Hence, Truncated Backpropagation through Time (TBPTT) is used to mitigate this issue by limiting the number of time steps for which gradients are backpropagated.

The key idea is to only backpropagate through a truncated sequence of time steps, rather than the entire sequence.



Truncated Backpropagation through Time (TBPTT)

Multilayer RNN is constructed by more than one hidden layer, where the output of one layer is fed as input to the next layer.

10.2 Image Captioning

First using CNN features from the image, then feeding them into an RNN to generate the caption.

Add the third weight matrix to combine the CNN features with the RNN hidden states.

before:

$$h = \tanh(W_{xh} * x + W_{hh} * h)$$

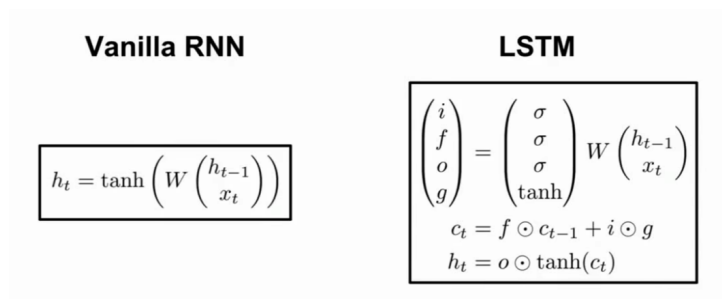
now:

$$h = \tanh(W_{xh} * x + W_{hh} * h + W_{ih} * v)$$

Image Captioning with CNN and RNN

Attention mechanisms can be used to improve the performance of image captioning models by allowing the model to focus on specific parts of the image when generating each word of the caption.

10.3 Long Short-Term Memory (LSTM)



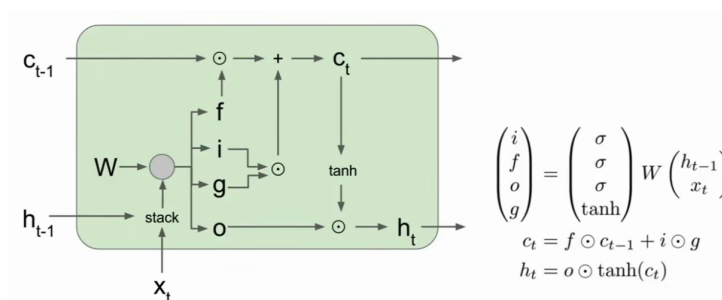
Long Short-Term Memory (LSTM)

four gates:

f: Forget gate, Whether to erase cell
i: Input gate, whether to write to cell
g: Gate gate (?), How much to write to cell
o: Output gate, How much to reveal cell

LSTM Gates

A figure to show the LSTM architecture, here the weight matrix is a conjunction of that of each gate: To be ware that here element wise multiplication is used other than the usual matrix multiplication.



LSTM figure

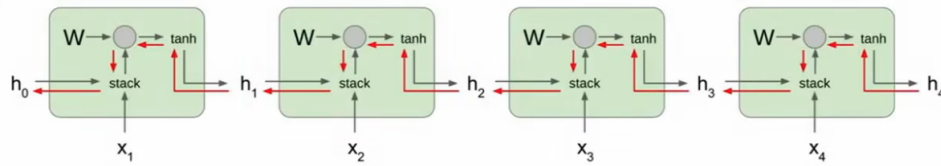
Gradient Flow

The gradient flow of the vanilla RNN:

Where computing gradient of h_0 involves many factors of W , which can lead to gradient explosion or vanishing.

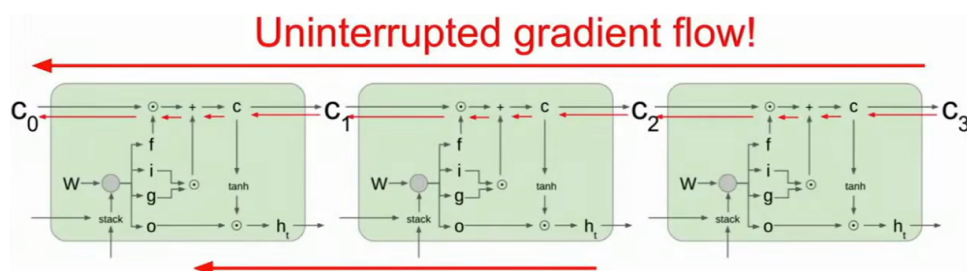
Vanilla RNN Gradient Flow

Bengio et al., "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994.
Pascanu et al., "On the difficulty of training recurrent neural networks", ICML, 2013



Gradient Flow in Vanilla RNN

And in LSTM, The gradient of weight matrix W will be calculated each time step by c (current cell state) and h (current hidden state) which are saved well during forward and backward.

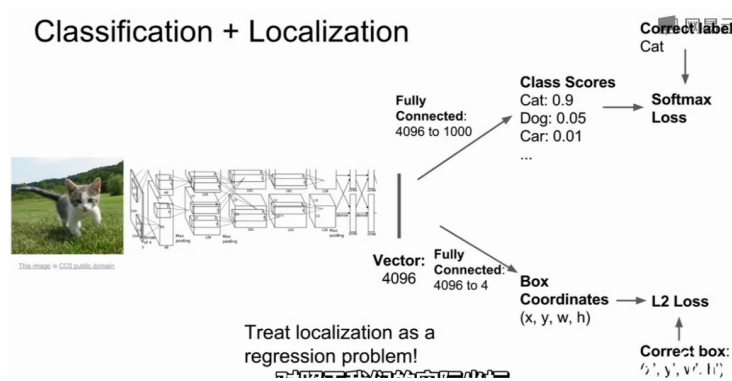


Gradient Flow in LSTM

Lecture 11: Segmentation, Localization and Detection

11.1 Classification and Localization

On the base of the output of scores, now additionally another output is produced for the bounding box coordinates(x, y, w, h), namely multi-task loss.



Classification and Localization

11.2 Object Detection

Different from regression since the number of objects is unknown and can vary from image to image.

- Sliding Window

The problem is that how to choose the window size and stride.

- Region Proposal

This approach generates many potential bounding boxes first(2000 for example) and then classifies them. It is a fixed algorithm **Selective Search** but not a learnable one.

1. R-CNN(Regions with CNN features)

- (a) Region Proposal: Using **Selective Search** to generate region proposals.
- (b) Feature Extraction: Extracting CNN features from each proposed region.
- (c) Classification: Classifying each proposed region using a set of SVMs.
- (d) Bounding Box Regression: Refining the bounding box coordinates using a regression model.

Problem: For 2000 region proposals, we need to perform 2000 forward passes through the network.

2. Fast R-CNN

Doing only one forward pass through the network.

- (a) Region Proposal: Using **Selective Search** to generate region proposals.
- (b) RoI Pooling: Applying RoI pooling to extract fixed-size feature maps for each proposed region.
- (c) Classification and Bounding Box Regression: Classifying each region and refining the bounding box coordinates using a single network.

Problem: The Selective Search algorithm is a CPU bottleneck and slows down the overall detection pipeline.

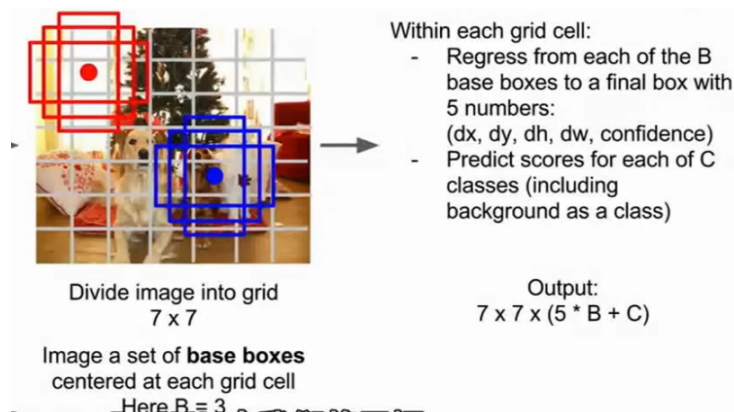
3. Faster R-CNN

Make CNN do proposals. Insert **Region Proposal Network (RPN)** before RoI Pooling.
Jointly train with 4 losses:

- (a) RPN classify object/not object.
- (b) RPN regress box coordinates.
- (c) Final classification score.
- (d) Final box coordinates.

- Detection without Proposals: YOLO(You Only Look Once) / SSD(Single Shot Detection)

Divides the image into a grids, generate a set of boxes centered at each grid cell.



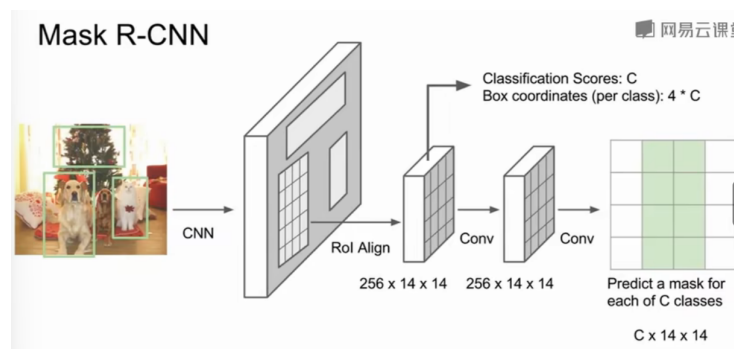
YOLO and SSD

11.3 Instance Segmentation

Like object detection, but predict a binary mask for each object, get pixel-wise segmentation.

- Mask R-CNN

Add a layer after fast R-CNN to predict a binary mask for each RoI.



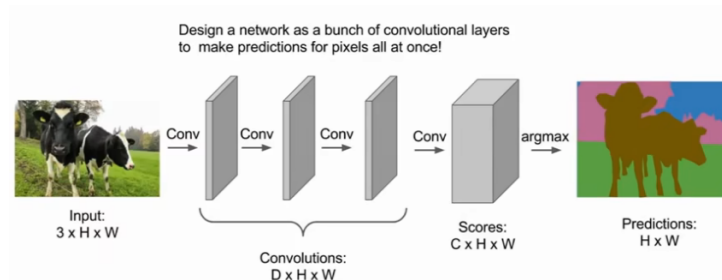
Mask R-CNN

11.4 Semantic Segmentation

To classify each pixel in the image, assigning a class label(here different objects are seen as different classes) to every pixel.

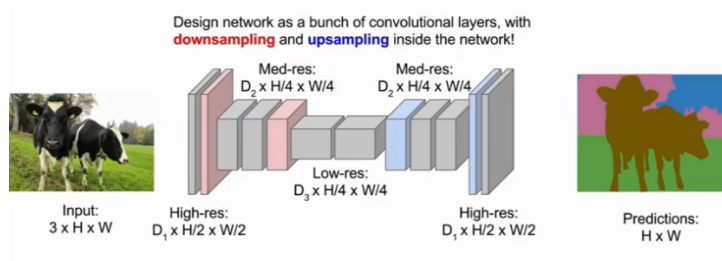
- Sliding Window: the complexity is high.
- Fully Convolutional Networks (FCN)

Replace the fully connected layers with convolutional layers, allowing the network to output a spatial map instead of a single label.



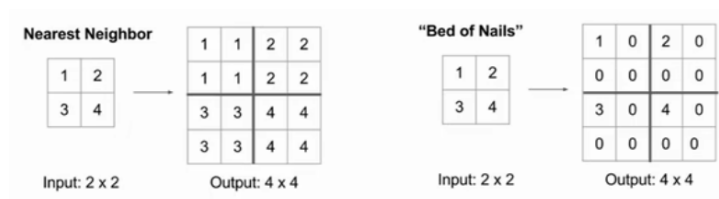
Fully Convolutional Networks

However, keep the size of the input image cost a lot. Hence downsampling and upsampling is often used(first decrease spatial resolution(清晰度), then increase in the second half):



Downsampling and Upsampling

– How to implement upsampling: by unpooling

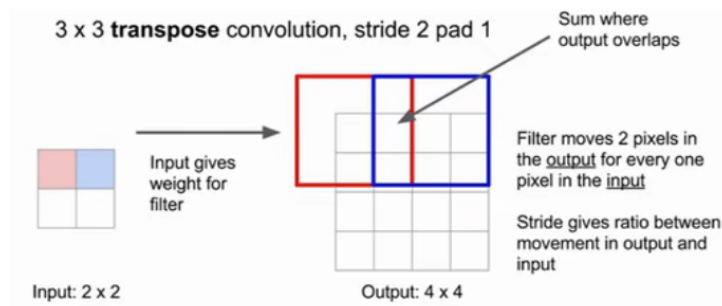


Unpooling

Remember which element was pooled in the downsampling(Pooling) step and reverse the operation during upsampling.

– Learnable Upsampling: Transpose(转置) Convolution

Seen the inputs as weights, to multiply with the filters and add together to get the output.



Transpose Convolution

Lecture 12: Visualizing and Understanding

12.1 Some Visualizations

1. First Layer: Visualize Filters
tend to show edges and textures.
2. The filters/kernels(raw weights) doesn't show interesting results
3. Last Layer: Nearest Neighbors
tend to be like the nearest neighbors in the training set.
t-SNE is often used for visualizing high-dimensional data in a lower-dimensional space.
4. Maximally Activating Patches:
Find the input patches that maximize the activation of a particular neuron to understand what features are being detected.
5. Occlusion(遮挡) Experiments:
Gradually occlude (遮挡) parts of the input image and observe the effect on the neuron's activation to identify important regions.
6. Saliency(显著性) Maps:
Compute the gradient of the output with respect to the input image to **identify important regions**.

12.2 Activating Neurons

- Visualizing CNN features: Gradient Ascent: Generate a synthetic image that maximally activates a neuron

$$I^* = \arg \max_I \boxed{f(I)} + \boxed{R(I)}$$

Neuron value Natural image regularizer

Gradient Ascent

1. Initialize image to zeros
- Repeat:**
2. Forward image to compute current scores
 3. Backprop to get gradient of neuron value with respect to image pixels
 4. Make a small update to the image
- Fooling Images/Adversarial Examples Generate an image that is classified as a target class by the network, but is unrecognizable to humans.
 1. Start from an arbitrary image
 2. Pick an arbitrary class

3. Modify the image to maximize the class
 4. Repeat until network is fooled
- DeepDream: Amplify existing features

Choose an image and a layer in a CNN, repeat:

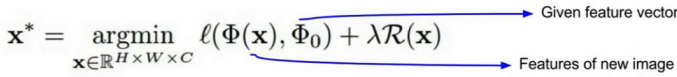
1. Forward: compute activations at chosen layer
2. Set gradient of chosen layer equal to its activation, which is equivalent to:

$$I^* = \arg \max_I \sum_i f_i(I)^2$$
3. Backward: Compute gradient on image
4. Update image

- Feature Inversion

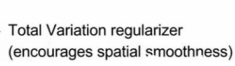
Given a CNN feature vector for an image, find a new image that matches the given feature vector and look natural.

$$\mathbf{x}^* = \underset{\mathbf{x} \in \mathbb{R}^{H \times W \times C}}{\operatorname{argmin}} \ell(\Phi(\mathbf{x}), \Phi_0) + \lambda \mathcal{R}(\mathbf{x})$$



$$\ell(\Phi(\mathbf{x}), \Phi_0) = \|\Phi(\mathbf{x}) - \Phi_0\|^2$$

$$\mathcal{R}_{V^2}(\mathbf{x}) = \sum_{i,j} \left((x_{i,j+1} - x_{ij})^2 + (x_{i+1,j} - x_{ij})^2 \right)^{\frac{\alpha}{2}}$$



Feature Inversion

Here a new regularization **Total Variation regularizer**(全变差正则化) is introduced to encourage spatial smoothness in the generated image.

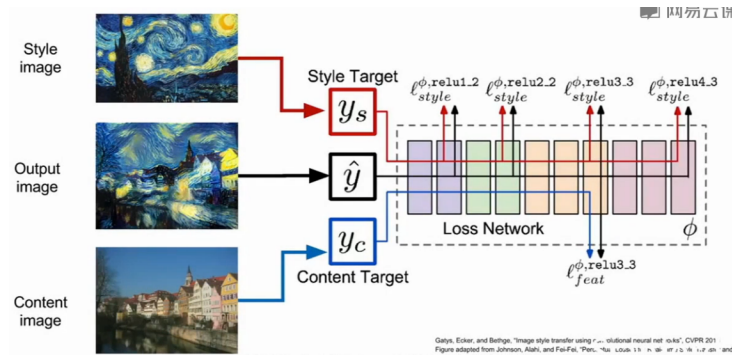
- Texture Synthesis

Given a texture image, generate a new image that has a similar texture. This can be done by optimizing an image to match the feature statistics of the texture image in a CNN.

- Traditional Methods: Nearest Neighbors
- Neural Texture Synthesis: Gram Matrix

For a $CxHxW$ feature map, reshape it to a $Cx(H * W)$ matrix F , then compute the Gram matrix $G = F * F^T$, which captures the correlations between different feature channels.

1. Pretrain a CNN on ImageNet
 2. Run input texture forward, record activations on every layer
 3. At each layer compute the Gram matrix
 4. Initialize generated image from random noise
- Repeat:
5. Pass generated image through the CNN, compute Gram matrix at each layer
 6. Compute loss: weighted sum of L2 distances between Gram matrices of generated image and texture image
 7. Backprop to get gradient on image
 8. Make gradient step on image



Neural Style Transfer

- Neural Style Transfer

Multiple style images: Mix style from multiple images by taking a weighted average of Gram matrices.

Problem: requires many forward / backward passes, very slow.

Possible Solution:

1. Train a feedforward network to approximate the style transfer process.
2. Use pretrained CNN to compute same losses as before.
3. After training, stylize images using a single forward pass.

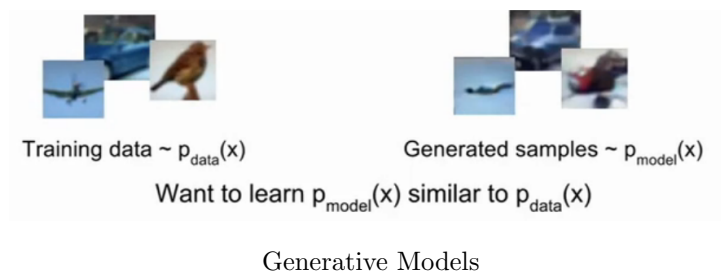
Lecture 13: Generative Models

13.1 Unsupervised Learning

Goal: Learn some underlying hidden structure of the data.

13.2 Generative Models

Given training data, generate new samples from same distribution.

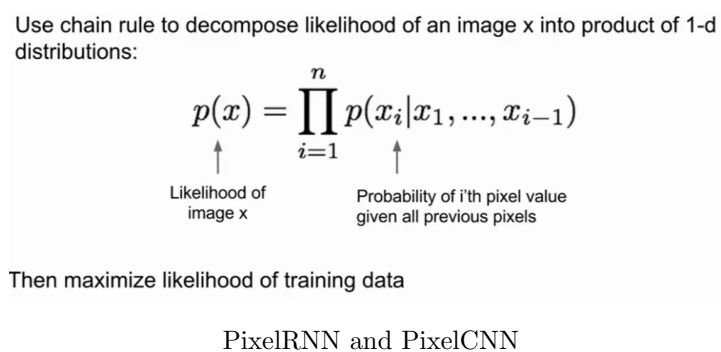


Several flavors:

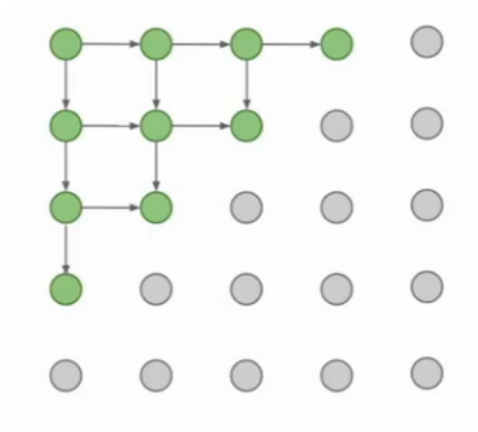
- Explicit density estimation: explicitly define and solve for $p_{model}(x)$
- Implicit density estimation: learn model that can sample from $p_{model}(x)$ without explicitly defining it.

1. PixelRNN and PixelCNN

Fully visible belief network, explicit density model.



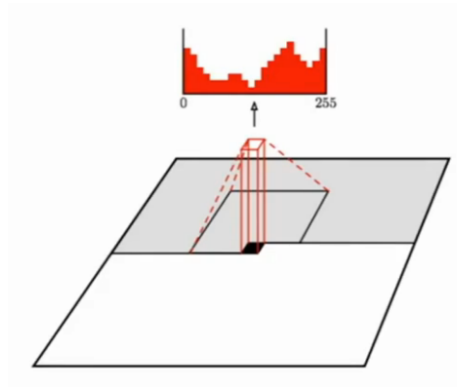
- PixelRNN
 - Generate image pixels starting from corner.
 - It is slow since it is sequential.



PixelRNN

- PixelCNN

Using CNN with masked convolutions(the filter multiplied with a mask) to learn the maximum likelihood of the pixel distribution.



PixelCNN

In the first layer, the mask is $\begin{bmatrix} 1 & 1 & 1 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$, and in subsequent layers: $\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$

By multiple layers, the receptive field increases, allowing the model to capture the context of the entire image.

What the model learns is the Maximum Likelihood Estimation (MLE) of the pixel distribution. Here in the training process, the output of the model is a probability distribution for each pixel.

The training pixel data is used in input, also to compute the Cross Entropy loss: $L = -\sum_i \log p_{\theta}(x_i | x_{<i})$

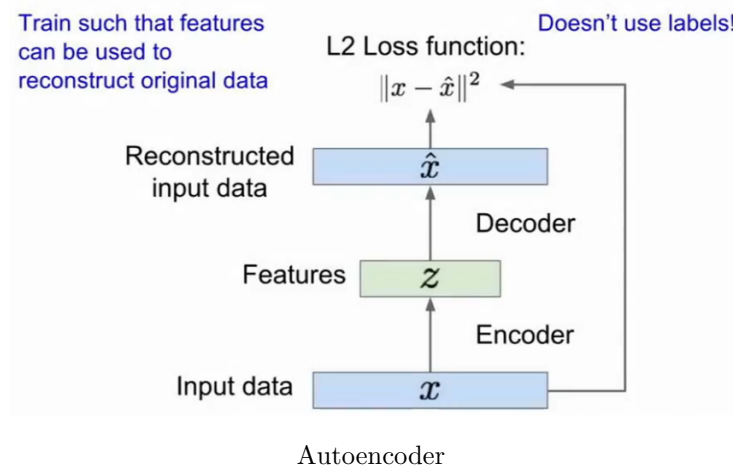
PixelCNN can trains in parallel across different pixels, making it faster than PixelRNN. However, generation must still proceed sequentially and is still slow.

2. Variational Autoencoders(VAE: 变分自编码器)

VAEs define intractable(难以计算, 不可求解) density function with latent z (潜在变量):

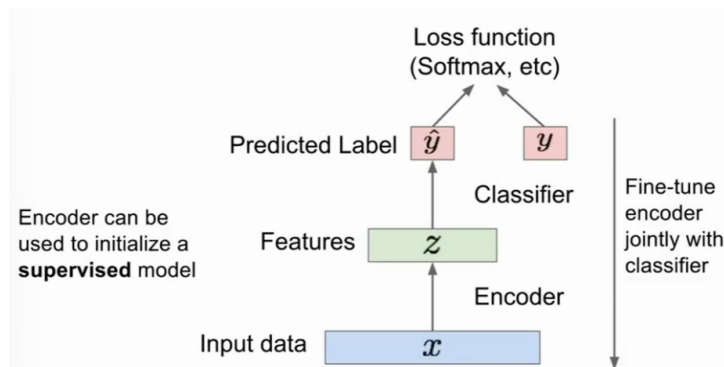
$$p_{\theta}(x) = \int p(z)p_{\theta}(x|z) dz$$

(a) Some background: Autoencoders



After training, throw away the decoder.

The encoder can be used to initialize a supervised model:



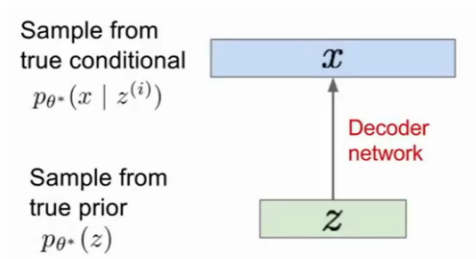
Using Encoder for Supervised Learning

(b) Variational Autoencoders(VAE: 变分自编码器)

Probabilistic spin(概率建模) on autoencoders: sample from the model to generate data.

The true parameters are θ .

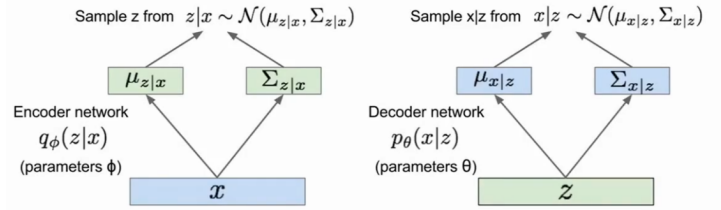
In VAEs, z and the output $\hat{x}$ are both a distribution.



Sampling from the VAE

To maximize the likelihood of the data which is intractable(derivation is difficult), in addition to decoder network modeling $p_{\theta}(x|z)$, **define additional encoder network $q_{\phi}(z|x)$ to**

approximate $p(z|x)$.



Variational Autoencoder

Now equipped with encoder and decoder networks, the maximum likelihood estimation can be performed using the reparameterization trick:

$$\begin{aligned}
 \log p_\theta(x^{(i)}) &= \mathbf{E}_{z \sim q_\phi(z|x^{(i)})} [\log p_\theta(x^{(i)})] \quad (p_\theta(x^{(i)}) \text{ Does not depend on } z) \\
 &= \mathbf{E}_z \left[\log \frac{p_\theta(x^{(i)} | z) p_\theta(z)}{p_\theta(z | x^{(i)})} \right] \quad (\text{Bayes' Rule}) \\
 &= \mathbf{E}_z \left[\log \frac{p_\theta(x^{(i)} | z) p_\theta(z) q_\phi(z | x^{(i)})}{p_\theta(z | x^{(i)}) q_\phi(z | x^{(i)})} \right] \quad (\text{Multiply by constant}) \\
 &= \mathbf{E}_z [\log p_\theta(x^{(i)} | z)] - \mathbf{E}_z \left[\log \frac{q_\phi(z | x^{(i)})}{p_\theta(z)} \right] + \mathbf{E}_z \left[\log \frac{q_\phi(z | x^{(i)})}{p_\theta(z | x^{(i)})} \right] \quad (\text{Logarithms}) \\
 &= \mathbf{E}_z [\log p_\theta(x^{(i)} | z)] - D_{KL}(q_\phi(z | x^{(i)}) || p_\theta(z)) + D_{KL}(q_\phi(z | x^{(i)}) || p_\theta(z | x^{(i)}))
 \end{aligned}$$

↑
Decoder network gives $p_\theta(x|z)$, can compute estimate of this term through sampling. (Sampling differentiable through reparam. trick, see paper.)

↑
This KL term (between Gaussians for encoder and z prior) has nice closed-form solution!

↑
 $p_\theta(z|x)$ intractable (saw earlier), can't compute this KL term :(But we know KL divergence always ≥ 0 .

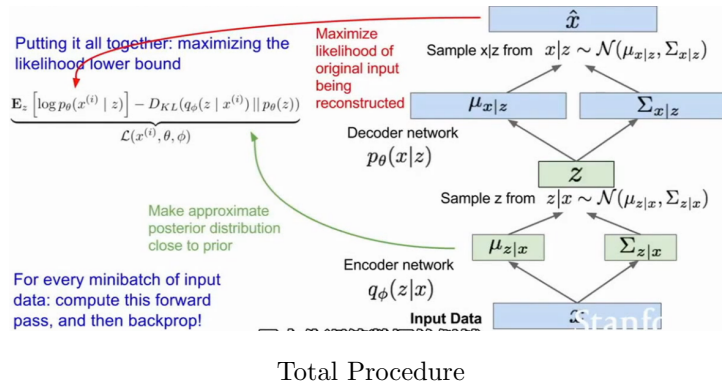
Reparameterization Trick

$$\begin{aligned}
 &= \underbrace{\mathbf{E}_z [\log p_\theta(x^{(i)} | z)] - D_{KL}(q_\phi(z | x^{(i)}) || p_\theta(z))}_{\mathcal{L}(x^{(i)}, \theta, \phi)} + \underbrace{D_{KL}(q_\phi(z | x^{(i)}) || p_\theta(z | x^{(i)}))}_{> 0} \\
 &\boxed{\log p_\theta(x^{(i)}) \geq \mathcal{L}(x^{(i)}, \theta, \phi)} \quad \boxed{\theta^*, \phi^* = \arg \max_{\theta, \phi} \sum_{i=1}^N \mathcal{L}(x^{(i)}, \theta, \phi)} \\
 &\text{Variational lower bound ("ELBO")} \quad \text{Training: Maximize lower bound}
 \end{aligned}$$

Lower Bound

In which the first term we maximize is to reconstruct the input data, and the second term is to make approximate posterior distribution close to the prior.

In total:

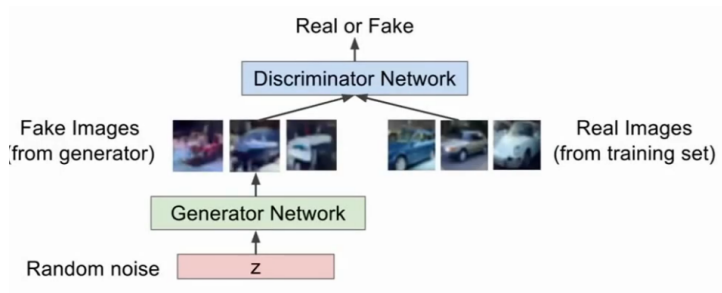


After training, only the decoder is needed. When generating, we can sample from the prior $p(z)$ and pass it through the decoder to obtain new data points.

3. Generative Adversarial(对抗) Networks(GAN)

GANs don't work with explicit density function.

GANs consist of two neural networks, a generator that try to fool the discriminator by generating real-looking images, and discriminator network that try to distinguish between real and fake images.



Train jointly in **minimax game**

Discriminator outputs likelihood in (0,1) of real image

Minimax objective function:

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

Discriminator output for real data x

Discriminator output for generated fake data $G(z)$

- Discriminator (θ_d) wants to **maximize objective** such that $D(x)$ is close to 1 (real) and $D(G(z))$ is close to 0 (fake)
- Generator (θ_g) wants to **minimize objective** such that $D(G(z))$ is close to 1 (discriminator is fooled into thinking generated $G(z)$ is real)

Minimax Game

Alternate between:

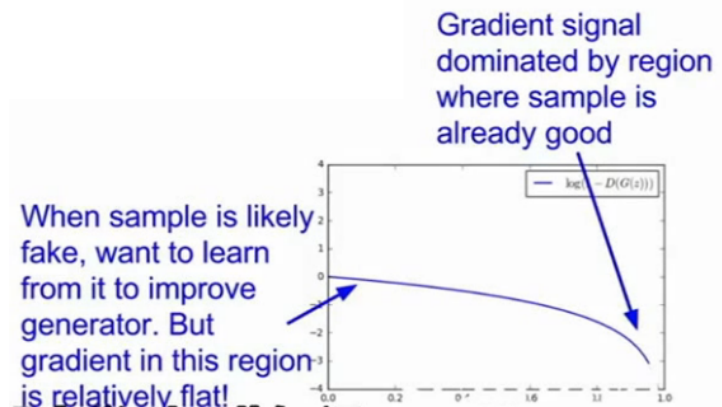
(a) **Gradient ascent** on discriminator

$$\max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

(b) **Gradient descent** on generator

$$\min_{\theta_g} \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z)))$$

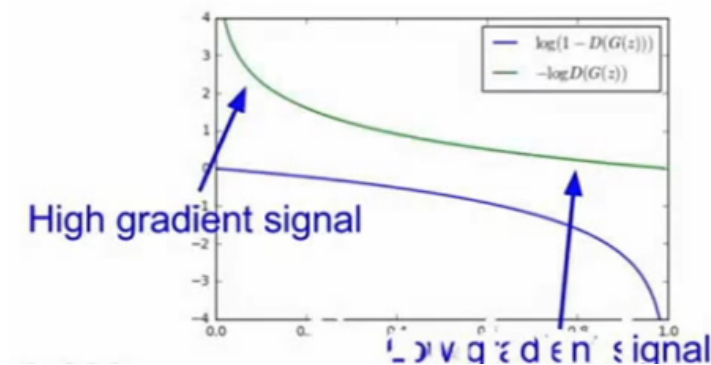
However, when optimizing this objective, we have higher gradient signal for good samples but low gradient signal for poor samples:



Problem with Gradient Descent

Hence, instead we do **Gradient ascent** on generator, to maximizing the likelihood of discriminator being wrong:

$$\max_{\theta_g} \mathbb{E}_{z \sim p(z)} \log D_{\theta_d}(G_{\theta_g}(z))$$



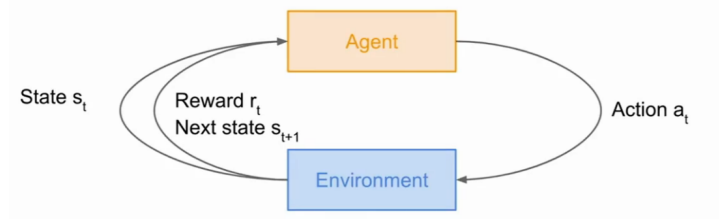
Different Objective for Generator

During every training iteration, we first update the discriminator and then the generator, alternatively.

After training, only the generator is needed to produce new samples.

Lecture 14: Reinforcement Learning

Problems involving an **agent** interacting with an **environment**, which provides numeric **reward** signals.



Reinforcement Learning Episode

14.1 Markov Decision Process (MDP)

1. MDP

Defined by: $(\mathcal{S}, \mathcal{A}, \mathcal{R}, \mathbb{P}, \gamma)$

$\mathcal{S}$: set of possible states

$\mathcal{A}$: set of possible actions

$\mathcal{R}$: distribution of reward given (state, action) pair

$\mathbb{P}$: transition probability i.e. distribution over next state given (state, action) pair

γ : discount factor

Markov Decision Process Definition

- At time step $t=0$, environment samples initial state $s_0 \sim p(s_0)$
- Then, for $t=0$ until done:
 - Agent selects action a_t
 - Environment samples reward $r_t \sim R(\cdot | s_t, a_t)$
 - Environment samples next state $s_{t+1} \sim P(\cdot | s_t, a_t)$
 - Agent receives reward r_t and next state s_{t+1}
- A policy π is a function from $\mathcal{S}$ to $\mathcal{A}$ that specifies what action to take in each state
- **Objective:** find policy π^* that maximizes cumulative discounted reward: $\sum_{t \geq 0} \gamma^t r_t$

Markov Decision Process

2. Some definitions:

- **value function:** the expected cumulative reward from following the policy from state s

$$V^\pi(s) = \mathbb{E} \left[\sum_{t \geq 0} \gamma^t r_t | s_0 = s, \pi \right]$$

- **Q-value function:** the expected cumulative reward from taking action a in state s and then following the policy

$$Q^\pi(s, a) = \mathbb{E} \left[\sum_{t \geq 0} \gamma^t r_t | s_0 = s, a_0 = a, \pi \right]$$

3. Bellman Equation:

$$Q^*(s, a) = \max_{\pi} \mathbb{E} \left[\sum_{t \geq 0} \gamma^t r_t | s_0 = s, a_0 = a, \pi \right] = \mathbb{E}_{s' \sim P(s'|s, a)} \left[r + \gamma \max_{a'} Q^*(s', a') | s, a \right]$$

Use this equation as an iterative update. However, the Q-value is not scalable, difficult to compute.

14.2 Q-learning

1. Q-learning

Use a function approximator to estimate the action-value function: $Q(s, a; \theta) \approx Q^*(s, a)$

And if the function approximator is a deep neural network -> **Deep Q-Network (DQN)**

Remember: want to find a Q-function that satisfies the Bellman Equation:

$$Q^*(s, a) = \mathbb{E}_{s' \sim \mathcal{E}} \left[r + \gamma \max_{a'} Q^*(s', a') | s, a \right]$$

Forward Pass

Loss function: $L_i(\theta_i) = \mathbb{E}_{s, a \sim \rho(\cdot)} [(y_i - Q(s, a; \theta_i))^2]$

where $y_i = \mathbb{E}_{s' \sim \mathcal{E}} \left[r + \gamma \max_{a'} Q(s', a'; \theta_{i-1}) | s, a \right]$

Backward Pass

Gradient update (with respect to Q-function parameters θ):

$$\nabla_{\theta_i} L_i(\theta_i) = \mathbb{E}_{s, a \sim \rho(\cdot); s' \sim \mathcal{E}} \left[(r + \gamma \max_{a'} Q(s', a'; \theta_{i-1}) - Q(s, a; \theta_i)) \nabla_{\theta_i} Q(s, a; \theta_i) \right]$$

Iteratively try to make the Q-value close to the target value (y_i) it should have, if Q-function corresponds to optimal Q^* (and optimal policy π^*)

Q-learning

2. Experience Replay

The current Q-network parameters determines next training samples which can lead to bad feedback loops.

Hence, continually update a **replay memory** table of transitions (s_t, a_t, r_t, s_{t+1}) as experience episodes.

Train Q-network on random minibatches of transitions from the replay memory, instead of consecutive samples, each transition can also contribute to multiple weight updates which brings greater data efficiency.

3. Policy Gradients

- Q-value is hard to compute when the action space is large or continuous.
- Q-learning doesn't learn the policy directly, but rather the action-value function.

For each policy, define its value: $J(\theta) = \mathbb{E} \left[\sum_{t \geq 0} \gamma^t r_t | \pi_{\theta} \right]$. We want to find $\theta^* = \arg \max_{\theta} J(\theta)$.

14.3 REINFORCE algorithm

1. REINFORCE algorithm

$$J(\theta) = \mathbb{E}_{\tau \sim p(\tau; \theta)} [r(\tau)] = \int_{\tau} r(\tau) p(\tau; \theta) d\tau$$

where $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots)$ is a trajectory.

We have: $p(\tau; \theta) = \prod_{t \geq 0} p(s_{t+1}|s_t, a_t) \pi_\theta(a_t|s_t)$

Thus: $\log p(\tau; \theta) = \sum_{t \geq 0} \log p(s_{t+1}|s_t, a_t) + \log \pi_\theta(a_t|s_t)$

And when differentiating: $\nabla_\theta \log p(\tau; \theta) = \sum_{t \geq 0} \nabla_\theta \log \pi_\theta(a_t|s_t)$ Doesn't depend on transition probabilities!

Therefore when sampling a trajectory τ , we can estimate $J(\theta)$ with

$$\nabla_\theta J(\theta) \approx \sum_{t \geq 0} r(\tau) \nabla_\theta \log \pi_\theta(a_t|s_t)$$

Trick

Now differentiate: $\nabla_\theta J(\theta) = \int_\tau r(\tau) \nabla_\theta p(\tau; \theta) d\tau$ which is intractable.

A trick can be used: $\nabla_\theta p(\tau; \theta) = p(\tau; \theta) \frac{\nabla_\theta p(\tau; \theta)}{p(\tau; \theta)} = p(\tau; \theta) \nabla_\theta \log p(\tau; \theta)$

Then:

$$\nabla_\theta J(\theta) = \int_\tau r(\tau) p(\tau; \theta) \nabla_\theta \log p(\tau; \theta) d\tau = \mathbb{E}_{\tau \sim p(\tau; \theta)} [r(\tau) \nabla_\theta \log p(\tau; \theta)]$$

now can estimate with Monte Carlo sampling.

As to the term $\nabla_\theta \log p(\tau; \theta)$, it is independent of the transition and can be estimated using the log-likelihood of the trajectory under the current policy:

According to the value of $r(\tau)$, change the probabilities of the actions seen.

However, a good trajectory leads to all its actions were good (the reward multiplied with all its action directly), this can lead to high variance in the gradient estimates, making training unstable.

2. Variance reduction

- First idea: Push up probabilities of an action seen, only by the cumulative future reward from that state:

$$\nabla_\theta J(\theta) \approx \sum_{t \geq 0} \left(\sum_{t' \geq t} r_{t'} \right) \nabla_\theta \log \pi_\theta(a_t|s_t)$$

- Second idea: Use discount factor γ to ignore delayed effects:

$$\nabla_\theta J(\theta) \approx \sum_{t \geq 0} \left(\sum_{t' \geq t} \gamma^{t'-t} r_{t'} \right) \nabla_\theta \log \pi_\theta(a_t|s_t)$$

- Third idea: Baseline

The raw value of a trajectory isn't necessarily meaningful, but whether a reward is better or worse than **what you expect to get**.

Introduce a baseline function dependent on the state only:

$$\nabla_\theta J(\theta) \approx \sum_{t \geq 0} \left(\sum_{t' \geq t} r_{t'} - b(s_t) \right) \nabla_\theta \log \pi_\theta(a_t|s_t)$$

How to choose a baseline:

- A simple baseline: constant moving average of rewards experiences so far from all trajectories.
- A better baseline: use the value function $V(s_t)$.

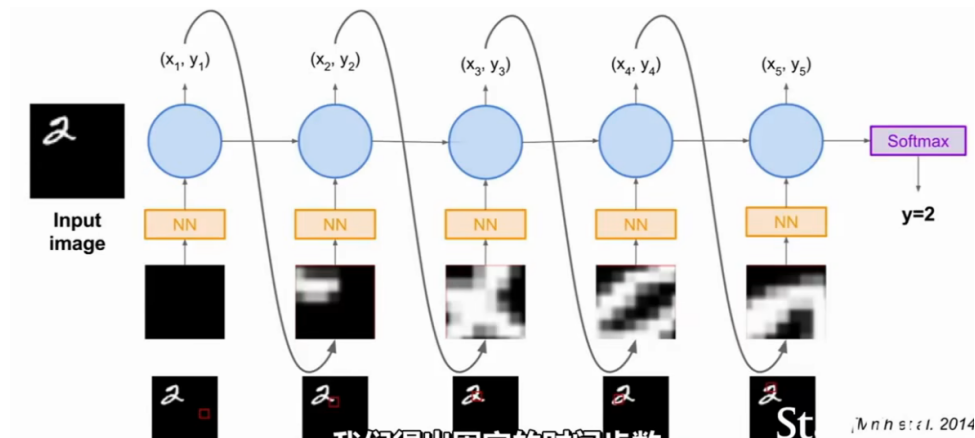
14.4 Actor-Critic Algorithm

- Combine value function and policy gradient methods.
- Use a critic to estimate the value function $V(s_t)$ and an actor to update the policy. Hence there's no need to wait for the final reward of a trajectory.

14.5 REINFORCE in action

1. Recurrent Attention Model(RAM):

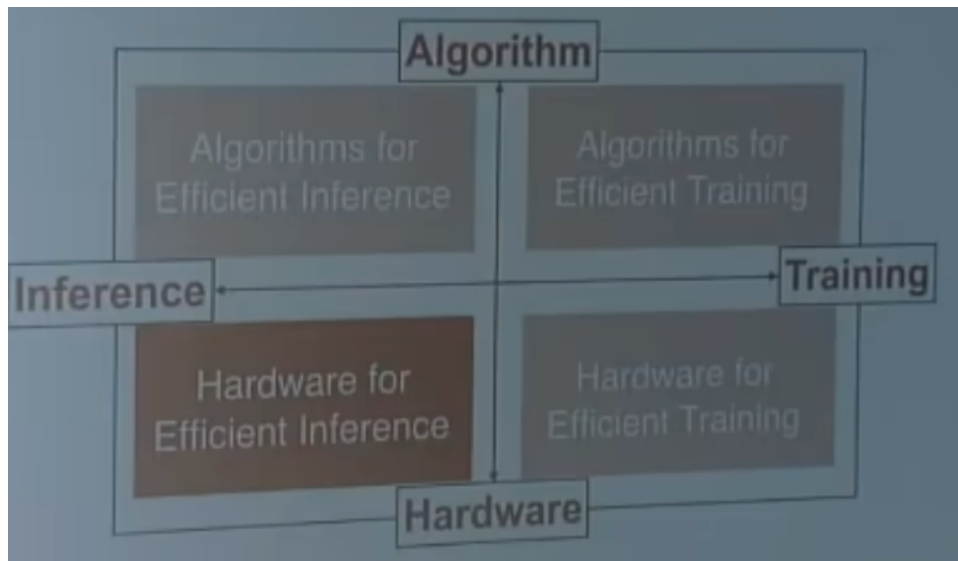
Using REINFORCE to select the most relevant parts of an image for a given task(glimpse).



Recurrent Attention Model

2. AlphaGO

Lecture 15: Efficient Inference and Training



Agenda

15.1 Part 1: Algorithms for Efficient Inference

1. Pruning: pruning neural network...
2. Weight Sharing: sharing weights between different layers or parts of the network to reduce the number of parameters
3. Quantization: quantizing the weight and activation
4. Low Rank Approximation: to reduce redundancy of high dimension data
5. Binary / Ternary Network: using binary or ternary weights and activations
6. Winograd Transformation: an algorithm for fast convolution

15.2 Part 2: Hardware for Efficient Inference

15.3 Part 3: Algorithms for Efficient Training

1. Parallelization
 - Data Parallelism: splitting the data across multiple GPUs
 - Model Parallelism: splitting the model across multiple GPUs
2. Mixed Precision with FP16 and FP32
3. Model Distillation(蒸馏): training a smaller model to mimic a larger model
4. DSD: Dense-Sparse-Dense Training: dense training + sparse pruning training + dense retraining

15.4 Part 4: Hardware for Efficient Training